

第7章 文件 I/O

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 node:fs 模块的三种风格:同步/回调/Promise,并理解生产选型
- 掌握文本读写与编码(UTF-8、BOM),二进制读写(Buffer)与 C 的 fread/fwrite 对照
- 掌握 path 模块:join/resolve、basename/dirname/extname,跨平台分隔符
- 掌握目录遍历、创建、删除与文件存在性判断
- 掌握大文件流式处理:createReadStream/readline,理解内存占用差异
- 掌握 JSON 序列化的坑:undefined、NaN、循环引用、toJSON
- 掌握标准流与命令行参数,完成日志解析统计工具实验

💡 从 C 到 JavaScript:本章主题如何迁移

C 的文件操作是 fopen/fread/fwrite/fclose 四件套,配合 fseek/ftell、strtok 逐行解析,每一步都要检查返回值,EOF 与错误全靠 errno 与 feof 判断。JS 的 node:fs 模块把全套映射过来,而且更进一步:每个操作都有同步、回调、Promise 三种形态,并配套了目录、权限、流式与 watch 等 C 标准库没有的高级能力。

最大的心智差异是异步:C 的 fread 是阻塞的,读完才返回;Node 的异步 readFile 立即返回一个 Promise,数据到了才回调。服务器场景必须用异步(否则一次磁盘读就把整个事件循环卡死),脚本场景可以用同步版本(init 阶段读配置)。第9章将深讲事件循环,本章先用"await 等结果"的姿势。

二进制视角也有变化:C 直接 fwrite 结构体(对齐、字节序、版本兼容都是坑);JS 用 Buffer 精确控制字节,配合 JSON/字符串化做跨版本兼容的序列化。文件不存在时,C 的 fopen 返回 NULL;JS 的异步 API 抛 ENOENT 错误,需要 try/catch 或 existsSync 预判。

最后,路径处理:C 的 "a/b" 拼接在 Windows 上就是事故;JS 用 path.join/path.resolve 自动处理分隔符,并用 path.win32/path.posix 保证跨平台一致。

📦 前置知识自查

- C 的 fopen/fclose/fread/fwrite/fprintf/fscanf 全套
- C 的文本模式与二进制模式("rb"/"wb")
- C 的 errno、feof、ferror 错误处理模式
- C 的 getline/strtok 逐行解析
- 第5章 Buffer 基础(字节与编码)

🚀 本章学习路线

1. **第一阶段(读写)**:读 §7.1–§7.2,跑通三种风格的读写示例
2. **第二阶段(路径目录)**:读 §7.3,练习 path 各 API 输出
3. **第三阶段(流与序列化)**:读 §7.4–§7.6,重点理解流式处理的内存优势
4. **第四阶段(实验)**:完成章末"日志解析与统计工具"
5. **验收标准**:能独立写出"读配置 → 处理 → 写结果"的完整异步流程,并处理 ENOENT

本章知识导图

- 第7章 文件 I/O
 - §7.1 基础读写:三种风格、编码与 BOM
 - §7.2 二进制 I/O:Buffer 读写、与 fread/fwrite 对照
 - §7.3 路径与目录:join/resolve、遍历、创建删除
 - §7.4 大文件流式处理:readline、内存对比
 - §7.5 序列化:JSON 的坑与安全做法
 - §7.6 标准流:stdin/stdout/stderr、命令行参数
 - §7.7 生产场景:配置文件、CSV、日志追加

§7.1 基础读写:同步、回调与 Promise

C	Node(fs 模块)	说明
fopen(path, "r")	fs.readFile	整个文件读入内存/回调
fopen(path, "w")	fs.writeFile	覆盖写
fopen(path, "a")	fs.appendFile	追加写
fread(buf, ...)	fs.read(fd, buf, ...)	低级读(带句柄)
fclose(fp)	fs.close(fd) / 自动关闭	readFile 全程自动管理
fseek/ftell	fs.readFileSync 的 position 参数	少见,生产基本用不着
feof	读空即 EOF,不报错	文件读完返回空串,不设错误

```

const fs = require('node:fs');

// 1. 同步风格:init 阶段/脚本工具可用(会阻塞事件循环)
fs.writeFileSync('/tmp/hello.txt', '你好,文件系统\n');
const text = fs.readFileSync('/tmp/hello.txt', 'utf8');
console.log('同步读:', text.trim());

// 2. 回调风格:老代码常见,链式回调易读难写
fs.readFile('/tmp/hello.txt', 'utf8', (err, data) => {
  if (err) {
    console.error('读失败:', err.message);
    return;
  }
  console.log('回调读:', data.trim());
});

// 3. Promise 风格:生产首选(与第9章 async/await 配套)
async function main() {
  await fs.promises.writeFile('/tmp/p.txt', 'Promise 写入\n');
  const data = await fs.promises.readFile('/tmp/p.txt', 'utf8');
  console.log('Promise 读:', data.trim());

  // 追加:对应 C 的 "a" 模式
  await fs.promises.appendFile('/tmp/p.txt', '追加一行\n');
  console.log('追加后:', (await fs.promises.readFile('/tmp/p.txt', 'utf8')).trim());
}
main().catch((err) => console.error('异步出错:', err.message));

```

⚠ 常见误区 7.1:readFile 的文件不存在与编码问题

文件不存在时,同步版抛 ENOENT 异常,回调版 `err.code === 'ENOENT'`,Promise 版 reject 带 code。生产防御: `fs.existsSync` 预判 + try/catch 兜底(存在与可读之间可能竞态,兜底不能省)。编码:不传 'utf8' 时 readFile 返回 Buffer;传了 'utf8' 但文件带 BOM 时,字符串开头会残留 U+FEFF 字符,需 `str.replace(/^/, '')` 去除。

§7.2 二进制 I/O:Buffer 与 fread/fwrite 对照

```
// 二进制写:与 C 的 fwrite(buf, size, n, fp) 对应
const data = Buffer.from([0x01, 0x02, 0xff, 0x00]);
fs.writeFileSync('/tmp/data.bin', data);
console.log('二进制已写:', data.toString('hex')); // 0102ff00

// 二进制读:不传编码直接得到 Buffer(不要用 utf8 解码二进制!)
const raw = fs.readFileSync('/tmp/data.bin');
console.log('二进制读回:', raw.toString('hex'));
console.log('字节数:', raw.length); // 4

// 结构化数据二进制定义:C 的 fwrite(struct) 是字节级;JS 用 DataView 精确布局
function writeRecord(path, records) {
  const ab = new ArrayBuffer(records.length * 8); // 每条:uint32 + float32
  const dv = new DataView(ab);
  records.forEach((r, i) => {
    dv.setUint32(i * 8, r.id); // 大端 uint32
    dv.setFloat32(i * 8 + 4, r.score); // 大端 float32
  });
  fs.writeFileSync(path, Buffer.from(ab));
}

function readRecords(path) {
  const buf = fs.readFileSync(path);
  const dv = new DataView(buf.buffer, buf.byteOffset, buf.byteLength);
  const out = [];
  for (let i = 0; i < dv.byteLength / 8; i++) {
    out.push({ id: dv.getUint32(i * 8), score: dv.getFloat32(i * 8 + 4) });
  }
  return out;
}

writeRecord('/tmp/rec.bin', [{ id: 1001, score: 87.5 }, { id: 1002, score: 92.25 }]);
console.log('结构读回:', JSON.stringify(readRecords('/tmp/rec.bin')));

// 低级句柄操作(对应 fopen/fclose):
const fd = fs.openSync('/tmp/fd.bin', 'w');
try {
  fs.writeSync(fd, Buffer.from('fd 写入'));
} finally {
  fs.closeSync(fd); // 第5章:句柄必须成对关闭
}
console.log(fs.readFileSync('/tmp/fd.bin', 'utf8')); // fd 写入
```

⚠ 常见误区 7.2:把二进制文件当 utf8 读

任何非文本文件(图片、压缩包、二进制协议)都不要传 'utf8' 读:字节被按 UTF-8 解码会得到乱码甚至被替换字符破坏。规则:文本文件传编码,二进制文件不传,拿到 Buffer 后用 DataView/位运算处理。

§7.3 路径与目录

```
const path = require('node:path');

// 1. 拼接与解析:C 的 "a/b" 手拼在 Windows 上是灾难
console.log(path.join('data', 'sub', 'file.txt')); // data\sub\file.txt(平台分隔符)
console.log(path.resolve('data', '..', 'conf.json')); // 绝对路径,.. 正常解析
console.log(path.basename('/a/b/c.txt'));           // c.txt
console.log(path.dirname('/a/b/c.txt'));             // /a/b
console.log(path.extname('/a/b/c.txt'));             // .txt

// 2. 跨平台:模块统一用 / 或 path.join;windows 上的盘符由 path 处理
// path.win32 与 path.posix 可以强制指定规则(如生成给别的系统用的路径)

// 3. 存在性、类型、信息
const p = '/tmp/test-path.txt';
fs.writeFileSync(p, 'x');
console.log(fs.existsSync(p));                       // true
console.log(fs.statSync(p).size);                    // 1,类似 C 的 fseek+ftell 取大小
console.log(fs.statSync(p).isFile());                // true
console.log(fs.statSync(p).isDirectory());           // false

// 4. 目录操作
fs.mkdirSync('/tmp/jsdir/sub', { recursive: true }); // 像 mkdir -p
fs.writeFileSync('/tmp/jsdir/sub/a.txt', 'A');
fs.writeFileSync('/tmp/jsdir/sub/b.txt', 'B');

// 5. 目录遍历(递归):替代 C 的 opendir/readdir 手写递归
function walk(dir, depth = 0) {
  const entries = fs.readdirSync(dir, { withFileTypes: true });
  for (const e of entries) {
    console.log(' '.repeat(depth) + e.name + (e.isDirectory() ? '/' : ''));
    if (e.isDirectory()) walk(path.join(dir, e.name), depth + 1);
  }
}
walk('/tmp/jsdir');

// 6. 删除
fs.rmSync('/tmp/jsdir', { recursive: true, force: true }); // 类似 rm -rf
console.log('删除后存在:', fs.existsSync('/tmp/jsdir'));    // false
```



工程建议 7.1:永远用 path.join 拼路径,禁止字符串拼接

'a' + '/' + 'b' 在 Windows 上可能得到 a/b(可接受)但在拼接用户输入时可能产生路径穿越(../)。统一 path.join/path.resolve;读取用户可控文件名时先 path.basename 剥离目录部分,再拼到白名单目录,这是文件服务的安全底线。

§7.4 大文件流式处理:内存优势

readFile 把整个文件读进内存:1GB 日志文件就吃 1GB+ 堆。流式处理一次只读一小块:内存占用与文件大小无关。Node 的 readline 可以逐行消费任意大小的文件。

```

const readline = require('node:readline');
const { createReadStream } = require('node:fs');

// 生成一个 1MB 的测试日志
async function genLog() {
  let lines = [];
  for (let i = 0; i < 20000; i++) {
    lines.push(`${Date.now() + i} [INFO] 请求 #${i} 完成 ${i % 7}ms`);
    if (lines.length > 5000) {
      await fs.promises.appendFile('/tmp/big.log', lines.join('\n') + '\n');
      lines = [];
    }
  }
  await fs.promises.appendFile('/tmp/big.log', lines.join('\n') + '\n');
}

// 流式逐行统计:内存占用恒定
async function analyze() {
  const rl = readline.createInterface({
    input: createReadStream('/tmp/big.log'),
    crlfDelay: Infinity, // 处理 \r\n
  });
  let count = 0;
  let totalMs = 0;
  for await (const line of rl) { // 逐行异步迭代
    const m = line.match(/完成 (\d+)ms/);
    if (m) {
      count++;
      totalMs += Number(m[1]);
    }
  }
  console.log(`总行数统计:${count} 条,平均耗时 ${((totalMs / count).toFixed(2))}ms`);
}

async function main() {
  await genLog();
  console.log('文件大小:', fs.statSync('/tmp/big.log').size, '字节');
  await analyze();
  console.log('内存占用:', Math.round(process.memoryUsage().heapUsed / 1024 / 1024), 'MB');
}
main().catch((e) => console.error(e));

```

★ 重点结论:流式处理三件套

① `createReadStream + readline`:逐行处理任意大文本(日志/CSV);② `createWriteStream`:分批写出,避免一次性拼大字符串;③ 管道 `pipe`:流到流的搬运,如 `readStream.pipe(writeStream)` 做文件复制,自动背压(内存不暴涨)。生产上传下载、日志切割都是流的天下。

```
// 流式复制(背压自动处理)
const src = createReadStream('/tmp/big.log');
const dst = fs.createWriteStream('/tmp/big-copy.log');
src.pipe(dst);
dst.on('finish', () => {
  console.log('复制完成,大小一致:', fs.statSync('/tmp/big-copy.log').size);
});
```

§7.5 序列化:JSON 与它的坑

C 的序列化靠 fwrite 结构体(脆弱:对齐、字节序、版本)或手写格式;JS 有原生 JSON。JSON 是现代 JS 程序的事实序列化格式:配置文件、HTTP 载荷、Redis 缓存全用它。但 JSON 有四个著名坑:

```

// 坑 1:undefined / 函数 / Symbol 属性被丢弃
const obj = { a: 1, b: undefined, c: () => {}, d: Symbol('s') };
console.log(JSON.stringify(obj)); // {"a":1} ← 三个都丢了!
// 数组里的 undefined 变成 null
console.log(JSON.stringify([1, undefined, 3])); // [1,null,3]

// 坑 2:NaN / Infinity 序列化 成 null
console.log(JSON.stringify({ x: NaN, y: Infinity })); // {"x":null,"y":null}

// 坑 3:循环引用报错
const cycle = { name: '环' };
cycle.self = cycle;
// console.log(JSON.stringify(cycle)); // ✖ TypeError: Converting circular structure

// 坑 4:BigInt 不支持
// console.log(JSON.stringify(1n)); // ✖ TypeError

// 坑 5:parse 的健壮性
const bad = '{"a":1,}';
try {
  JSON.parse(bad);
} catch (e) {
  console.log('JSON.parse 报错:', e.message); // Unexpected token
}

// 生产方案:toJSON 钩子自定义序列化
class Order {
  constructor(id, total) {
    this.id = id;
    this.total = total;
  }
  toJSON() { // stringify 自动调用
    return { id: this.id, total: this.total, currency: 'CNY' };
  }
}
console.log(JSON.stringify(new Order(1001, 99.5)));
// {"id":1001,"total":99.5,"currency":"CNY"}

// BigInt 兜底:转字符串传输
const big = { orderId: 9007199254740993n };
console.log(JSON.stringify(big, (k, v) => (typeof v === 'bigint' ? v.toString() : v)));
// {"orderId":"9007199254740993"}

```

⚠ 常见误区 7.3:JSON.parse 不校验直接当对象用

JSON.parse 返回的可能是 null、数字、数组,不只是对象: JSON.parse('null') 得到 null。解析后先验证形状再取字段: if (data && typeof data === 'object' && Array.isArray(data.list)) ,否则 undefined 一路传播,报错在十行之外,难排查。生产推荐用 zod/joi 这类校验库做运行时 schema 校验。

§7.6 标准流与命令行参数

```
// 标准流三件套:与 C 的 stdin/stdout/stderr 一一对应
process.stdout.write('写到标准输出\n');    // 相当于 printf
process.stderr.write('写到标准错误\n');    // 相当于 fprintf(stderr, ...)
console.error('console.error 也是 stderr');

// 读取 stdin 整行输入(脚本场景)
const readline = require('node:readline/promises');
async function ask() {
  const rl = readline.createInterface({ input: process.stdin, output: process.stdout });
  const name = await rl.question('你的名字:');
  console.log('你好,', name);
  rl.close();
}

// 命令行参数:C 的 argc/argv → process.argv
// node tool.js a b c
console.log('argv[0](node):', process.argv[0]);
console.log('argv[1](脚本):', process.argv[1]);
console.log('参数列表:', process.argv.slice(2)); // [ 'a', 'b', 'c' ]

// 生产参数解析:手写或 minimist/commander
function parseArgs(argv) {
  const out = { help: false, port: 3000 };
  for (let i = 0; i < argv.length; i++) {
    if (argv[i] === '--port') out.port = Number(argv[i + 1]);
    if (argv[i] === '--help') out.help = true;
  }
  return out;
}
console.log(parseArgs(['--port', '8080'])); // { help: false, port: 8080 }

// 退出码:C 的 return 0/非0 → process.exitCode
process.exitCode = 0;    // 成功;非 0 表示失败(CI 据此判断)
```

§7.7 生产场景:配置、CSV 与日志

```
// 1. 配置文件:JSON 配置读取 + 默认值合并
async function loadConfig(path) {
  let file = {};
  if (fs.existsSync(path)) {
    file = JSON.parse(await fs.promises.readFile(path, 'utf8'));
  }
  return {
    host: file.host ?? '0.0.0.0',    // ?? 保留合法空值
    port: file.port ?? 3000,
    debug: file.debug ?? false,
  };
}

// 2. CSV 简易处理:逗号分隔 + 引号剥离(生产用 csv-parse 库)
function parseCsvLine(line) {
  return line.split(',').map((f) => f.trim().replace(/^"|"$/g, ''));
}

// 3. 日志追加:appendFile 天然支持并发追加(C 的 "a" 模式)
async function logLine(level, msg) {
  const line = `[${new Date().toISOString()}] [${level}] ${msg}\n`;
  await fs.promises.appendFile('/tmp/app.log', line, 'utf8');
}

// 4. 主流程:读配置 → 解析 CSV → 记录日志 → 输出统计
async function main() {
  const cfg = await loadConfig('/tmp/app-config.json');
  console.log('配置:', JSON.stringify(cfg));

  fs.writeFileSync('/tmp/students.csv', 'name,score\n"李雷",88\n"韩梅梅",95\n');
  const rows = (await fs.promises.readFile('/tmp/students.csv', 'utf8'))
    .trim().split('\n').map(parseCsvLine);
  console.log('CSV 行数:', rows.length - 1);    // 去掉表头

  await logLine('INFO', `配置加载完成,端口 ${cfg.port}`);
  await logLine('INFO', `CSV 解析 ${rows.length - 1} 条记录`);
  const log = await fs.promises.readFile('/tmp/app.log', 'utf8');
  console.log('日志内容:\n' + log.trim());
}

main().catch((e) => console.error('主流程失败:', e));
```

工程建议 7.2:日志不是 console.log 就完事

生产日志三要求:① 结构化(JSON 行,便于 ELK/Loki 检索);② 分级(debug/info/warn/error,环境变量控制级别);③ 轮转(按大小/日期切割,用 pino/winston 或系统 logrotate)。console.log 只适合开发期与演示代码,进生产前替换为日志库。

本章速查表

主题	速查要点
三种风格	Sync(脚本/init)、回调(老代码)、fs.promises(生产首选)
写文件	writeFile 覆盖;appendFile 追加;都自动处理打开/关闭
读文本	readFile(path, 'utf8');带 BOM 时去
读二进制	不传编码 → Buffer;禁止 utf8 解二进制
句柄操作	openSync/readSync/writeSync/closeSync 成对,finally 关闭
文件不存在	ENOENT;existsSync 预判 + try/catch 兜底
路径	path.join 拼接;resolve 绝对化;basename/dirname/extname 拆分
目录	mkdirSync recursive;readdirSync withFileTypes;rmSync recursive
大文件	createReadStream + readline 逐行;pipe 复制;内存与文件大小无关
JSON 序列化	undefined/函数/Symbol 丢弃;NaN→null;循环引用抛错;BigInt 不支持
toJSON	自定义序列化钩子,stringify 自动调用
JSON 解析	parse 前检查形状;非法 JSON 抛 SyntaxError
标准流	stdout 输出 / stderr 错误 / stdin 输入;argv[0..1] 是 node 与脚本
退出码	process.exitCode = 0/非0,CI 依据
配置	JSON 配置 + ?? 默认值合并
日志	结构化 JSON 行、分级、轮转;pino/winston

最小必会清单

- 能写出 fs.promises 的读/写/追加三连,并处理 ENOENT
- 能说出三种风格(fs Sync/回调/Promise)的适用场景
- 能写出"文本带编码读、二进制不带编码读"的规则
- 能写出 path.join/resolve/basename/extname 的示例
- 能递归遍历目录并统计文件数
- 能用 createReadStream + readline 逐行统计 1GB 文件
- 能说出 JSON 序列化的四个坑与 toJSON 的用途
- 能写出 JSON.parse 前的形状校验
- 能说出 process.argv 与 C 的 argc/argv 的对应
- 能写出"读配置→处理→写结果→记日志"的完整异步流程

自测题

Q 1. fs.readFileSync(path, 'utf8') 与不传 'utf8' 有什么区别?对二进制文件应怎么读?

✅ 参考答案

传 'utf8' 返回解码后的字符串;不传返回 Buffer。二进制文件不传编码,拿到 Buffer 后用 DataView/位运算处理。

Q 2. `JSON.stringify({a: undefined, b: NaN, c: () => {}})` 输出什么?循环引用呢?

✅ 参考答案

`'{"b":null}'`。undefined 与函数属性被丢弃,NaN 变 null;循环引用抛 TypeError。

Q 3. 为什么大文件要用流式处理?readFile 读 1GB 文件会怎样?

✅ 参考答案

readFile 把整个文件读进内存,1GB 文件占用 1GB+ 堆,可能 OOM;流式一次只读一小块,内存占用恒定,还能逐行处理。

Q 4. `path.join('/a', '../b')` 与 `path.resolve('/a', '../b')` 分别得到什么?

✅ 参考答案

join 是纯拼接规范化:`./a/../b' → '/b'`;resolve 基于当前工作目录生成绝对路径。join 不保证绝对路径,resolve 保证。

Q 5. 读一个 UTF-8 带 BOM 的文件,字符串开头会有什么?怎么处理?

✅ 参考答案

开头有 U+FEFF(BOM 字符)。用 `str.replace(/^/, '')` 去除。

Q 6. 写出 `process.argv` 在 `node tool.js a b` 时的内容,以及如何取到业务参数。

✅ 参考答案

`['node的路径', 'tool.js的路径', 'a', 'b']`。业务参数是 `process.argv.slice(2)`,即 `['a', 'b']`。

章末实验题:日志解析与统计工具

实验 7:日志解析与统计工具(logstats)

实验目标:实现命令行工具:解析访问日志(CSV 风格),统计每行各字段,输出汇总报告与 JSON 结果文件,覆盖全章知识点。

实验要求:

- 任务 1(覆盖:读写、编码):读取日志文件(文本 + 二进制两种输入),支持 UTF-8 与 BOM 处理
- 任务 2(覆盖:流式处理、readline):用 createReadStream + readline 逐行解析,支持任意大小文件
- 任务 3(覆盖:路径与目录):输出目录用 path.join 拼接;--out-dir 参数控制;目录不存在时 mkdir recursive
- 任务 4(覆盖:JSON 序列化与坑):统计结果序列化到 result.json(注意 undefined 字段丢弃问题,先清洗再序列化)
- 任务 5(覆盖:标准流与参数):参数解析用 --level/--out-dir 风格;错误写 stderr;退出码非 0
- 任务 6(覆盖:appendFile 日志):工具自身的运行日志追加到 logs/tool.log(INFO/ERROR 分级)
- 任务 7(覆盖:存在性判断):文件不存在给出友好提示(ENOENT 处理)

实验环境:Node.js 20+,运行 `node logstats.js --file /tmp/access.log --level ERROR`。

第一步 写解析器:每行格式 "时间戳,级别,消息,耗时ms",逗号分隔解析为对象

第二步 写统计器:按级别计数、总耗时均值、最慢请求 TOP3

第三步 后写主流程:参数解析 → 校验文件 → 流式统计 → 写 result.json + tool.log

✅ 参考答案要点

核心:readline 逐行流式解析 + 统计聚合 + JSON 输出 + 参数/退出码规范。约 120 行。

```
// logstats.js — 运行:node logstats.js --file /tmp/access.log [--level INFO]
'use strict';
const fs = require('node:fs');
const path = require('node:path');
const readline = require('node:readline');
const { createReadStream } = require('node:fs');

// 1. 参数解析(覆盖:process.argv、Number 转换)
function parseArgs(argv) {
  const out = { file: null, level: null, outDir: 'out' };
  for (let i = 0; i < argv.length; i++) {
    if (argv[i] === '--file') out.file = argv[i + 1];
    if (argv[i] === '--level') out.level = String(argv[i + 1]).toUpperCase();
    if (argv[i] === '--out-dir') out.outDir = argv[i + 1];
    if (argv[i] === '--help') out.help = true;
  }
  return out;
}

// 2. 单行解析:时间戳,级别,消息,耗时ms(覆盖:split、trim、Number)
function parseLine(line) {
  const parts = line.split(',');
  if (parts.length < 4) return null;
  return {
    ts: parts[0].trim(),
    level: parts[1].trim(),
    msg: parts[2].trim(),
    cost: Number(parts[3].trim()), // 可能 NaN,统计时过滤
  };
}

// 3. 流式统计(覆盖:createReadStream、readline、for await)
async function analyzeFile(filePath, levelFilter) {
  const rl = readline.createInterface({
    input: createReadStream(filePath),
    crlfDelay: Infinity,
  });
  const byLevel = {};
  const costs = [];
  let total = 0;
  for await (const line of rl) {
    const rec = parseLine(line);
    if (rec === null) continue;
    total++;
    byLevel[rec.level] = (byLevel[rec.level] ?? 0) + 1;
    if (Number.isFinite(rec.cost)) {
      costs.push({ cost: rec.cost, msg: rec.msg });
    }
  }
  const sorted = costs.slice().sort((a, b) => b.cost - a.cost);
  const filtered = levelFilter ? total : total;
  const sum = costs.reduce((a, c) => a + c.cost, 0);
  return {
    totalLines: total,
    matchedLines: filtered,
  };
}
```

```

    byLevel,
    avgCostMs: costs.length > 0 ? Math.round((sum / costs.length) * 100) / 100 : 0,
    top3: sorted.slice(0, 3).map((c) => ({ cost: c.cost, msg: c.msg })),
  });
}

// 4. 工具日志(覆盖:appendFile、分级、时间戳)
async function toolLog(level, msg) {
  const dir = path.join(process.cwd(), 'logs');
  fs.mkdirSync(dir, { recursive: true });
  const line = `[${new Date().toISOString()}] [${level}] ${msg}\n`;
  await fs.promises.appendFile(path.join(dir, 'tool.log'), line, 'utf8');
}

// 5. 输出 JSON(覆盖:JSON 序列化、undefined 清洗)
async function writeResult(stats, outDir) {
  fs.mkdirSync(outDir, { recursive: true });
  const clean = {
    totalLines: stats.totalLines,
    byLevel: stats.byLevel,
    avgCostMs: stats.avgCostMs,
    top3: stats.top3,
  };
  await fs.promises.writeFile(
    path.join(outDir, 'result.json'),
    JSON.stringify(clean, null, 2),
    'utf8'
  );
}

// 6. 主流程(覆盖:存在性、ENOENT、stderr、退出码)
async function main() {
  const args = parseArgs(process.argv.slice(2));
  if (args.help || args.file === null) {
    console.error('用法:node logstats.js --file <日志文件> [--level 级别] [--out-dir 目录]');
    process.exitCode = 1;
    return;
  }
  if (!fs.existsSync(args.file)) {
    console.error(`错误:文件不存在:${args.file}`); // 走 stderr
    process.exitCode = 2;
    await toolLog('ERROR', `文件不存在:${args.file}`);
    return;
  }

  const stats = await analyzeFile(args.file, args.level);
  await writeResult(stats, args.outDir);
  await toolLog('INFO', `分析完成:${stats.totalLines} 行,输出到 ${args.outDir}/result.json`);

  console.log('==== 统计报告 =====');
  console.log('总行数:', stats.totalLines);
  console.log('级别分布:', JSON.stringify(stats.byLevel));
  console.log('平均耗时:', stats.avgCostMs, 'ms');
  console.log('最慢请求 TOP3:');
  for (const t of stats.top3) {

```



```
    console.log(` ${t.cost}ms ${t.msg}`);
  }
  console.log('结果文件:', path.join(args.outDir, 'result.json'));
}

main().catch((err) => {
  console.error('程序异常:', err.message);
  process.exitCode = 3;
});
```

设计说明:流式逐行解析保证大文件内存恒定;级别分布用对象计数(?? 初始化);耗时统计先过滤 `Number.isFinite` 排除 `NaN` 行;输出 JSON 前把可能 `undefined` 的字段清洗成确定结构;工具自身日志走 `appendFile` 分级落盘;错误统一走 `stderr` + 非零退出码,符合 CI 与脚本调用约定。可以先手写一个含 5 行的 `access.log` 测试,再生成 2 万行大文件验证内存表现。

下一章预告:第8章 网络通信 —— net 模块的 TCP/UDP、fetch 客户端、http.createServer 最小服务器,以及粘包/拆包与优雅关闭的生产坑。